Case Management Module — Design Specification

Version: 1.0 | **Date:** 2026-07-28 | **Status:** Draft

1. Overview

The Case Management module tracks the full lifecycle of civil and criminal cases handled by the law firm. It replaces the physical diary/tarikh register with a digital system that combines complete case lifecycle tracking with daily hearing (tarikh) management.

1.1 Core Capabilities

- Create and manage civil and criminal cases through their full lifecycle stages
- Track parties (plaintiff, defendant, accused, appellant) with automatic duplicate matching against existing client records
- Record and manage court hearing dates (tarikh) with calendar integration
- Role-based visibility: advocates see their cases, firm admin sees all, partners see firm-wide reports, paralegals support filings
- Timeline view showing every event in a case's history
- Auto-suggested linking of parties to existing client profiles

1.2 Scope — Phase 1

- Case types: Civil + Criminal only (extensible later)
- Hearing/calendar management
- Party matching/deduplication
- Case timeline
- Full CRUD with RBAC

1.3 Out of Scope (Phase 1)

- Document generation (templates, auto-fill)
- Billing integration with cases
- Court integration (e-filing APIs)
- Advanced analytics/dashboards

2. Entity Model

2.1 Entity Relationship

```
cases (1) ——— (N) hearings
cases (1) ——— (N) case_parties
cases (1) ——— (N) case_timeline
case_parties (N) ——— (1) client_profiles (optional)
case_parties (N) ——— (1) employee_profiles (optional, for advocates)
```

2.2 Case Entity

Single table with `case_type` discriminator (CIVIL/CRIMINAL). Common fields shared by both types. Type-specific fields are nullable and validated in service layer.

Common fields: id, firm_id, case_number, case_type, title, case_stage, status, court_name, court_case_number, judge_name, filing_date, filing_number, assigned_to (primary advocate), description

Civil-specific: mediation_date, mediation_outcome, written_statement_deadline

Criminal-specific: fir_number, fir_date, police_station, investigation_authority, arrest_date, charge_sheet_date, bail_status

2.3 Case Stages — Civil

`FILED → UNDER_SUMMONS → RESPONSE_PENDING → MEDIATION → EVIDENCE → ARGUMENT → JUDGMENT_AWAITED → JUDGMENT_DELIVERED → (APPEAL → CLOSED | EXECUTION → CLOSED)`

2.4 Case Stages — Criminal

`FIR_REGISTERED → UNDER_INVESTIGATION → CHARGE_SHEET_FILED → PLEA → TRIAL → JUDGMENT_AWAITED → JUDGMENT_DELIVERED → SENTENCING → (APPEAL → CLOSED | CLOSED)`

Stage transitions are validated. Invalid transitions return 400 BusinessRuleException.

2.5 CaseParty Entity

Tracks parties involved in a case with link to existing client records.

Fields: id, firm_id, case_id, party_type (PLAINTIFF/DEFENDANT/ACCUSED/APPELLANT/RESPONDENT/APPLICANT), representation (REPRESENTED/OPPOSING/SELF), full_name, mobile_no, email, address, client_id (FK, nullable), is_our_client, advocate_id (FK, nullable), notes

2.6 Hearing Entity

The core calendar/tarikh record.

Fields: id, firm_id, case_id, title, date, time, end_time, court_room, judge_name, hearing_type (FIRST_HEARING/PLEA/EVIDENCE/ARGUMENT/JUDGMENT/STATUS_CONF/OTHER), status (SCHEDULED/HELD/CANCELED/ADJOURNED), outcome, notes, attendees, created_by, advocate_id

2.7 CaseTimelineEvent Entity

Immutable log of significant events. Used for timeline view and audit trail.

Fields: id, firm_id, case_id, event_type (STAGE_CHANGE/HEARING_SCHEDULED/HEARING_HELD/HEARING_ADJOURNED/DOCUMENT_UPLOADED/PARTY_ADDED/JUDGMENT_RECORDED/NOT title, description, related_entity_id, created_at

---

3. Party Matching System

3.1 Matching Logic

```
User enters: name, phone, email
  -> Search client_profiles AND case_parties within same firm
     where: full_name (case-insensitive contains)
         OR mobile_no (exact match)
         OR email (exact match)
  -> Return ranked matches:
     HIGH:   name + phone match, or name + email match
     MEDIUM: exact name match only
     LOW:    partial name match only
  -> User can: link to existing, create new, or skip
```

3.2 API

`POST /api/v1/firm/parties/match` — Match party against existing records

3.3 Auto-suggestion on Case Create

When creating a case, the server performs matching server-side and returns potential matches in the response. Frontend prompts user to link or proceed.

---

4. Calendar Integration

4.1 Architecture

Calendar is a **view** over Hearing records. Hearings are the source of truth.

- Daily/weekly/monthly views filtered by advocate, date range, hearing type
- Today's hearings summary (dashboard widget)
- Conflict detection (same advocate, overlapping time → 409 Conflict)

4.2 Calendar API Endpoints

Method	Path	Description
GET	/api/v1/firm/calendar	Calendar view in date range
GET	/api/v1/firm/calendar/today	Today's hearings
GET	/api/v1/firm/calendar/upcoming	Upcoming hearings (next N days)

---

5. API Endpoints

5.1 Case CRUD

Method	Path	Permission
POST	/api/v1/firm/cases	CASE_MANAGEMENT:CREATE
GET	/api/v1/firm/cases	CASE_MANAGEMENT:VIEW
GET	/api/v1/firm/cases/{id}	CASE_MANAGEMENT:VIEW
PUT	/api/v1/firm/cases/{id}	CASE_MANAGEMENT:EDIT
DELETE	/api/v1/firm/cases/{id}	CASE_MANAGEMENT:DELETE
PUT	/api/v1/firm/cases/{id}/stage	CASE_MANAGEMENT:UPDATE_STATUS

5.2 Case Parties

Method	Path	Permission

GET	/api/v1/firm/cases/{id}/parties	CASE_MANAGEMENT:VIEW
POST	/api/v1/firm/cases/{id}/parties	CASE_MANAGEMENT:EDIT
PUT	/api/v1/firm/cases/{id}/parties/{partyId}	CASE_MANAGEMENT:EDIT
PUT	/api/v1/firm/cases/{id}/parties/{partyId}/link	CASE_MANAGEMENT:EDIT
DELETE	/api/v1/firm/cases/{id}/parties/{partyId}	CASE_MANAGEMENT:EDIT

5.3 Hearings & Calendar

Method	Path	Description
POST	/api/v1/firm/cases/{id}/hearings	Schedule hearing
GET	/api/v1/firm/cases/{id}/hearings	List case hearings
PUT	/api/v1/firm/hearings/{id}	Update hearing
DELETE	/api/v1/firm/hearings/{id}	Cancel/delete hearing
GET	/api/v1/firm/calendar/today	Today's hearings
GET	/api/v1/firm/cases/{id}/timeline	Full case timeline

6. Permissions & RBAC

6.1 Permission Definitions

Code	Action	Scope
CASE_MANAGEMENT:ACCESS	ACCESS	TENANT
CASE_MANAGEMENT:VIEW	VIEW	TENANT
CASE_MANAGEMENT:CREATE	CREATE	TENANT
CASE_MANAGEMENT:EDIT	EDIT	TENANT
CASE_MANAGEMENT:DELETE	DELETE	TENANT
CASE_MANAGEMENT:ASSIGN	ASSIGN	TENANT
CASE_MANAGEMENT:UPDATE_STATUS	UPDATE_STATUS	TENANT
CASE_MANAGEMENT:ARCHIVE	ARCHIVE	TENANT

6.2 Role Defaults

Role	Permissions
FIRM_ADMIN	All 8 permissions
ADVOCATE	ACCESS, VIEW, CREATE, EDIT, UPDATE_STATUS
PARALEGAL	ACCESS, VIEW, CREATE, EDIT

7. Integration with Existing Modules

- **Document Management:** Case-document junction table, DOCUMENT_UPLOADED timeline event
- **Client Management:** CaseParty.clientId links to ClientProfile, party matching searches client_profiles
- **Audit:** All mutations go through @Audit annotation; AuditEntity.CASE and AuditEntity.HEARING added
- **Employee:** CaseParty.advocateId links to EmployeeProfile for advocate assignment

8. Database Indexes

Table	Index	Purpose
cases	(firm_id, case_type)	Filter by type
cases	(firm_id, case_stage)	Pipeline view
cases	(firm_id, filing_date DESC)	Recent cases
cases	(firm_id, case_number) UNIQUE	Case lookup
cases	(firm_id, assigned_to)	Advocate caseload
case_parties	(case_id)	Parties by case
case_parties	(firm_id, full_name)	Party matching
case_parties	(client_id)	Client-linked cases
hearings	(case_id)	Hearings by case
hearings	(firm_id, date DESC)	Calendar view
hearings	(advocate_id, date DESC)	Advocate calendar
case_timeline	(case_id, created_at DESC)	Timeline display

8.1 Case Number Format

`{TYPE}-{YEAR}-{SEQUENCE:05d}` e.g. CIV-2026-00001, CRM-2026-00001. Generated per firm/year/type using a counter table.

9. Error Handling

Scenario	HTTP	Error Code
Invalid stage transition	400	INVALID_STAGE_TRANSITION
Hearing time conflict	409	HEARING_TIME_CONFLICT
Case not found	404	CASE_NOT_FOUND
Party not in firm	403	PARTY_NOT_IN_FIRM
Delete case with hearings	400	CASE_HAS_HEARINGS
Invalid stage for type	400	INVALID_STAGE_FOR_TYPE

10. Package Structure

```
com.lawfirm.erp.modules.casemanagement
  entity/     — Case.java, CaseParty.java, Hearing.java, CaseTimelineEvent.java, enums/
  repository/ — CaseRepository.java, CasePartyRepository.java, HearingRepository.java, CaseTimelineRepository.java
  service/    — CaseService.java, CasePartyService.java, HearingService.java, CalendarService.java, PartyMatchService.java,
CaseNumberGenerator.java
  controller/ — CaseController.java, CasePartyController.java, HearingController.java, CalendarController.java
  dto/        — request/ (9 DTOs), response/ (7 DTOs)
```

11. Implementation Order

1. Enums (CaseType, CaseStage, CaseStatus, PartyType, etc.)
2. Entities (Case, CaseParty, Hearing, CaseTimelineEvent)
3. Repositories (with custom queries)
4. DTOs (request/response classes)
5. CaseNumberGenerator service
6. PartyMatchService
7. CaseService (CRUD + stage transitions + timeline)
8. HearingService (CRUD + conflict detection)
9. CalendarService (calendar view queries)
10. CasePartyService
11. Controllers + Security config
12. DataInitializer update (CASE_MANAGEMENT permissions)
13. Tests

12. Testing Strategy

Unit Tests: CaseService (CRUD, stage transitions, duplicates), HearingService (schedule, conflicts), PartyMatchService (matching confidence), CalendarService (queries), CaseNumberGenerator (sequence)

Integration Tests: Full case creation with party matching, calendar with cross-case hearings, stage lifecycle validation, permission enforcement by role